

|                    |                                                                    |
|--------------------|--------------------------------------------------------------------|
| <b>Challenge:</b>  | The Pensieve Does Not Forget                                       |
| <b>Category:</b>   | Network Forensics / Cryptography / Steganography                   |
| <b>Difficulty:</b> | Hard                                                               |
| <b>Tools Used:</b> | Wireshark, foremost, hexed.it (Hex Editor), dcode.fr Steganography |

## Summary

A raw packet capture file disguised as a .bin file contains multiple layers of deception — fake traffic, embedded archives, hidden passwords, and encrypted text. You need to carve out a hidden ZIP from inside the network capture, find the password buried in the file's own header, decrypt an encrypted file using the correct cipher variant and finally extract the flag from the bottom. Every step has a decoy designed to send you the wrong way.

## Problem Statement

In the wizarding world, a Pensieve stores memories as silver liquid — extracted, literal, replayable. You don't read a Pensieve. You fall into it. You experience what happened. SNAPES captured memory the same way. Not a log file. Not a report. A raw packet capture - every byte that crossed the internal wire on the night FIENDFYRE triggered. They called it the Pensieve Dump. The file you recovered from the portrait was not the destination - it was merely the path that led you here. This artifact is deceptive. It is not a single structure, nor a single layer. What appears visible is only the surface. The truth lies deeper - buried beneath what is meant to distract, hidden where only careful observation reveals it. The archive is still locked. Enter the memory.

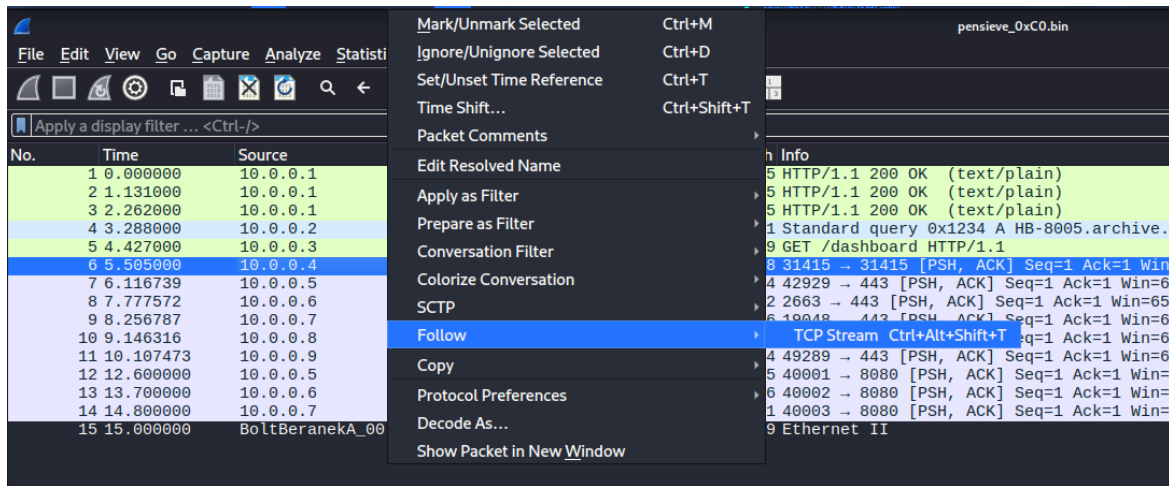
## Steps to Solve

1. Download the file from the drive link provided in the challenge.  
You'll get pensieve\_0xC0.bin
2. The file is named .bin which tells you nothing. In forensics you never trust the extension - always check what a file actually is before assuming anything.
3. Run: file pensieve\_0xC0.bin

```
(ghostrider_12@kali)-[~/Desktop]
$ file pensieve_0xC0.bin
pensieve_0xC0.bin: pcap capture file, microsecond ts (little-endian) - version 2.4 (Ethernet, capture length 65535)
```

Output: The file is identified as a PCAP file - a network packet capture.  
Open it in Wireshark : terminal command => wireshark pensieve\_0xC0.bin

4. Inside Wireshark, inspect the TCP streams. Go to Analyze → Follow → TCP Stream and cycle through them.



##### 5. What you'll find:

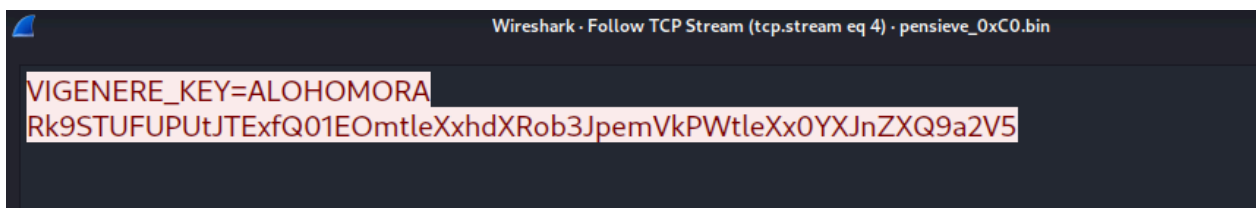
Mostly normal-looking HTTP traffic — noise

Several streams containing base64-like strings — traps

A DNS entry: HB-8005.archive.local — decoy

A cookie value: c2VjcmV0 which decodes to secret — decoy

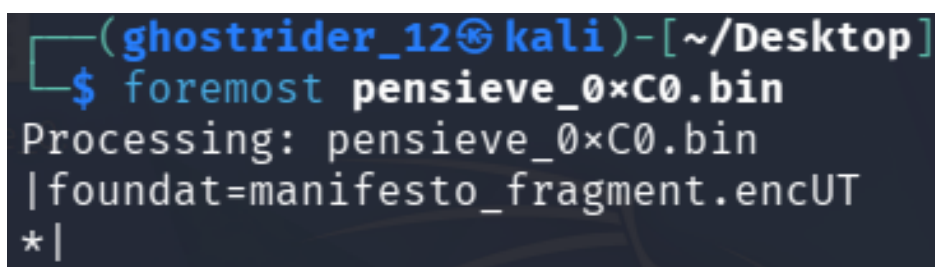
A string: VIGENERE\_KEY = ALOHOMORA — save this for later



##### 6. The traffic is deliberately designed to distract. Don't chase the base64 strings or the DNS entry, they are dead ends placed there to waste your time.

Problem statement connection: "What appears visible is only the surface." - everything you can read in the packets is surface level. The real data is elsewhere

##### 7. Since the real data isn't inside any packet, something must be physically embedded inside the binary file itself.



##### 8. Run foremost: foremost pensieve\_0xC0.bin

##### 9. Navigate the output:

cd output

ls

cd zip

ls

```

(ghostrider_12@kali)-[~/Desktop]
$ cd output

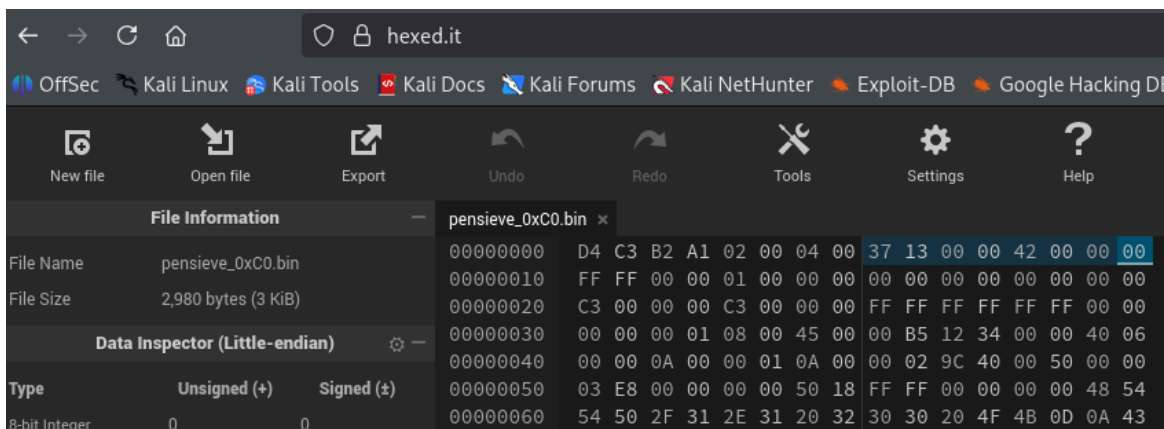
(ghostrider_12@kali)-[~/Desktop/output]
$ ls
audit.txt  zip

(ghostrider_12@kali)-[~/Desktop/output]
$ cd zip

(ghostrider_12@kali)-[~/Desktop/output/zip]
$ ls
00000003.zip

```

10. What you find: A ZIP file - 00000003.zip - containing manifesto\_fragment.enc.
11. Try to extract it:  
unzip 00000003.zip
12. It asks for a password. You don't have it yet.
13. Hint 1 connection: Hint 1 told you the real data was physically embedded - not transmitted in the packets. Foremost proved that by carving the ZIP directly out of the binary.
14. A PCAP file has a fixed global header at the very start of the file with several fields. Two of those fields are normally set to zero but here they contain unusual values:  
thiszone - timezone offset field (normally 0)  
sigfigs - significant figures field (normally 0)
15. Open pensieve\_0xC0.bin in a hex editor like HxD. The global PCAP header starts at byte 0. Navigate to bytes 8–11 for thiszone and bytes 12–15 for sigfigs.
16. Opening the file in a hex editor shows a valid PCAP header.



Since PCAP uses little-endian encoding, these values must be reversed:

thiszone → 0x1337

sigfigs → 0x42

17. These two values combined form the password: 1337\_42
18. Hint 2 connection: Hint 2 told you the password was stamped into the fabric of the file itself — not in any packet. The PCAP header fields thiszone and sigfigs were the exact location it pointed at.
19. Now unzip using the password: unzip 00000003.zip
20. Enter password: 1337\_42  
ls  
cat manifesto\_fragment.enc

21. You'll see a block of encrypted text - scrambled letters that look almost readable but aren't quite.

```
(ghostrider_12@kali)-[~/Desktop/output/zip]
$ cat manifesto_fragment.enc
IHTDXSW_ZNAWK // ZBTKANAA XDMYXO // YPLWPGHGNX

Wakmkz qnajt. Vaot gz tmp dtbg vkw Odbzwtxt mv Zobgk hri zhku
widgsnf gujvozjrmouv k rryhwp vy uenazke nsqnji. Jgognke jwjaqlu.
Cdfwzkuv bali. Olizmdm wgykqolgssh lhm. Zryqlidl. Txrnizgovil.
Zmpi. Vak lutwjt oqk zay nazke gz oec mgjzmeog jwjaql lkpagtk ua
yjmsydow ximdix pop krka oaik.

Z nsggh hek pqvno hekj ovk zwpagui. E samhh kmkvq gsnh ac gt.
G vai sawl ev vai gaq vfk Fsndwoxo. G qwpdkmkj vkah gaq vfxnw choqw.

Thji st jqogcnnh xbd ah vkw wqgekzmn ytlgu. Vfkaw auk ohvkn omuk.
Cabdde hek fhmg. Hth ezdb-enc st oo vfk nnx.

-- TW // AN-8005

Thnjw dw h zmywyj vt sfo y hthq dxmwn.
Nms kmkvqytsyi eguoclwlp - azdt etlv fovxjwi pkzifv.

Gmcldoq ew cgndwakj jjjis.
Vquth mmpaalw zkuh.

Eeoo xicrsnt gp byv zspbfk.
Ev ji ixcdvfgu zrxzkj, wyspa nqkmvksnf ndbioyt.

Sg qtu qoeh hx uulixzhayl lhm ytnv hzli,
xnax yao vfk vmjiw - gut skah hofh ybn ymtvp.

[Osljh Nxvd]
Vfk pmozoul qoz pmfidl ybpw, gyldx thn qwn:
EZDB_MFX
```

22. From Wireshark earlier you found VIGENERE\_KEY = ALOHOMORA.

Go to [dcode.fr](https://dcode.fr) → Vigenère Cipher, enter the encrypted text with key ALOHOMORA.

Result: A mix of readable words and complete gibberish. The key is correct but the cipher variant is wrong.

The screenshot shows the dcode.fr Vigenère Cipher tool. On the left, a search bar has 'ALO HOMORA' entered, and the results list 'ALO HOMORA' with a link to the tool. The main interface is the 'VIGENERE DECODER' section. It has a 'VIGENERE CIPHERTEXT' field containing the encrypted text from the previous block. Below it, 'PLAINTEXT LANGUAGE' is set to 'English' and 'ALPHABET' is 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'. The 'DECRYPTION METHOD' section has 'KNOWING THE KEY/PASSWORD' selected with the key 'ALO HOMORA'. The 'AUTOMATIC DECRYPTION' button is highlighted.

23. The Beaufort cipher is a variant of Vigenère where instead of adding the key to the plaintext, you subtract the plaintext from the key. It looks identical to Vigenère from the outside — the only sign something is wrong is the partial gibberish result you just got.

24. Go to [dcode.fr](https://dcode.fr) → Beaufort Cipher, enter the same encrypted text with key ALOHOMORA.

25. Output: Fully readable decrypted text. Scroll through it carefully all the way to the bottom.

Search for a tool

SEARCH A TOOL ON DCODE

e.g. type 'sudoku'

BROWSE THE FULL DCODE TOOLS' LIST

Results

ALO HOMORA

ABCDEFGHIJKLMNOPQRSTUVWXYZ (26)

SEVERUS\_SNAPE // INTERNAL RECORD // CLASSIFIED

Eleven years. That is how long the Ministry of Magic has been selling infrastructure access to unnamed buyers. Citizen records. Movement logs. Magical vulnerability data. Packaged. Transmitted. Sold. The buyers are not named in any official record because no official record was ever made.

I built the system they are selling. I wrote every line of it. I was told it was for the Ministry. I believed that for three years.

This is fragment one of the evidence chain. There are three more. Follow the chain. The kill-key is at the end.

-- SS // HB-8005

There is a pattern to what they erase. Not everything disappears -- only what

### BEAUFORT CIPHER

Cryptography > Poly-Alphabetic Cipher > Beaufort Cipher

#### BEAUFORT DECODER

BEAUFORT CIPHERTEXT

0kl13j1480  
WDWAUKZV\_KWD||3  
(ZAQWAAAXJH{va3\_z3zw13w3\_i4e\_4ad\_oh3\_b13w})

MODE ☒ BEAUFORT CLASSIC (KEY-TXT)  
☐ GERMAN VARIANT (TXT-KEY)

LANGUAGE English

ALPHABET ABCDEFGHIJKLMNOPQRSTUVWXYZ

DECRYPT AUTOMATICALLY

#### DECRYPTION METHOD

☒ WITH THE CIPHER KEY: ALOHOMORA  
☐ WITH THE KEY-LENGTH: 6  
☐ KNOWING A PORTION OF THE KEY (JOKER=?): K?Y  
☐ KNOWING A PLAINTEXT WORD: CODE  
☐ BEAUFORT CRYPTANALYSIS (KASISKI TEST + IC)

DECRYPT

See also: [Vigenere Cipher](#) — [Caesar Cipher](#)

#### BEAUFORT ENCODER

BEAUFORT PLAIN TEXT

dCode Beaufort

MODE ☒ BEAUFORT CLASSIC (KEY-TXT)  
☐ GERMAN VARIANT (TXT-KEY)

CIPHER KEY KEY

26. Hint 3 connection: Hint 3 told you that standard Vigenère would give partial gibberish and that the correct cipher was Beaufort. The partial gibberish from your first attempt confirmed exactly what the hint warned about.

27. The decrypted output contains readable text - but don't stop there. After several empty lines at the bottom you will find the flag.

28. Hint 4 connection: Hint 4 told you to scroll past the visible text into the blank space at the bottom. Without this hint, most people would stop at the readable decrypted text and miss the flag entirely.

29. Flag : MORSMORDRE{th3p3ns13v3\_s4w\_4ll\_th3\_l13s}